Project Requirements & Acceptance Specification

End-to-End Customer Churn Prediction System | Version 1.0 | 24 October 2026

Version: 1.0 | **Prepared:** 24 October 2026 | **Status:** Implementation handover package

1. Purpose

Build a reproducible, production-style machine-learning system that estimates customer churn risk and helps a retention team prioritize customer outreach. The project is an educational demonstration using the IBM Telco Customer Churn dataset; it is not a production decision system without further validation, governance, privacy review, and business testing.

2. Scope

Included: data understanding, data cleaning, stratified 70/15/15 split, preprocessing, baseline and tree-model comparison, threshold selection, test evaluation, explainability, batch scoring, FastAPI endpoints, Streamlit UI, Docker packaging, unit tests, and a basic feature-drift report. Supporting deliverables include notebooks, requirement traceability, executive summary, slides, demo script, retention recommendations, and monitoring runbook.

3. Functional requirements

| ID | Requirement | Acceptance evidence |

|---|---|---|

| FR-01 | Load a Telco churn CSV with a `Churn` target. | `data/raw/churn.csv`; EDA notebook and report |

| FR-02 | Summarize shape, data types, missingness, unique values and target distribution. | Notebook 01; EDA artifacts |

| FR-03 | Produce 6–10 meaningful univariate/bivariate visuals and written insights. | Notebook 01; `reports/figures/` |

| FR-04 | Clean column names and handle numeric/categorical missing values without target leakage. | Notebook 02; reusable preprocessing pipeline |

| FR-05 | Create reproducible stratified 70/15/15 train/validation/test splits. | Notebook 02; `data/processed/` outputs |

| FR-06 | Encode categorical features and scale numeric features where appropriate. | `src/features/build_features.py`; notebook 03 |

| FR-07 | Train a Logistic Regression baseline and report ROC-AUC, PR-AUC, F1, precision and recall. | Notebook 03; metrics artifacts |

| FR-08 | Compare Random Forest and XGBoost and tune model settings/threshold on validation data. | `src/models/train.py`; notebook 04; comparison CSV |

| FR-09 | Keep the test split isolated until model selection and threshold selection are complete. | Training workflow and notebook narrative |

| FR-10 | Persist the selected model and threshold with joblib. | `models/churn_model.joblib` generated locally |

| FR-11 | Produce at least four explainability visuals and at least three actionable retention strategies. | Notebook 05; figures; `docs/RETENTION_STRATEGIES.md` |

| FR-12 | Score a CSV batch and save probabilities/labels. | `src/pipeline/batch_score.py`; batch prediction CSV |

| FR-13 | Provide FastAPI `/health` and `/predict` endpoints. | `src/serving/app.py`; `/docs`; tests |

| FR-14 | Provide Streamlit form with churn risk/probability output. | `app_ui/streamlit_app.py`; local UI |

| FR-15 | Compare current inference features with training reference data and produce HTML/CSV drift reports. | `src/monitoring/drift_report.py`; monitoring report |

| FR-16 | Provide a weekly monitoring schedule/runbook. | `docs/MONITORING_RUNBOOK.md`; `docs/weekly_drift_check.ps1` |

| FR-17 | Document setup, data source, commands, assumptions and limitations. | `README.md`; this specification |

| FR-18 | Provide project presentation and a 2–3 minute demonstration script. | `slides/capstone_presentation.pptx`; `docs/DEMO_SCRIPT.md` |

4. Non-functional requirements

- **Reproducibility:** fixed random seeds and documented split strategy.
- **Maintainability:** separate modules for preprocessing, training, serving, monitoring and tests.
- **Usability:** browser-based Streamlit form and interactive API docs.
- **Portability:** Python virtual environment and Docker image instructions.

- **Safety/privacy:** no real customer personal data or secrets in source control; sample outputs are for demonstration only.
- **Interpretability:** explanations are descriptive associations, not causal claims.
- **Performance:** acceptable for a classroom/demo-sized CSV; no production SLA is claimed.

5. Data and model assumptions

- The expected target column is `Churn`, usually Yes/No in the IBM Telco dataset.
- `customerID` is an identifier and must not be a model feature.
- Blank `TotalCharges` values may appear as whitespace and should be coerced to missing.
- Numeric missing values are median-imputed; categorical missing values are filled using a training-fitted imputer and encoded with unknown-category handling.
- Use stratified random splitting when no reliable time field exists. The recommended split is 70% train, 15% validation, 15% test.
- Choose the model and probability threshold using validation data; evaluate the locked choice on test data once.

6. Model evaluation

Report ROC-AUC, average precision (PR-AUC), precision, recall, F1, confusion matrix and a precision-recall curve. Churn is a minority class in the standard Telco dataset, so do not use accuracy alone. Threshold selection should reflect the cost of missed churners versus unnecessary retention outreach.

7. Acceptance checklist

- [ ] Data dictionary and 6–10 meaningful EDA visuals with observations.
- [ ] Reproducible 70/15/15 splits and cleaning rationale.
- [ ] Baseline notebook with confusion matrix and PR curve.
- [ ] Model comparison and validation-based threshold selection.
- [ ] Saved model artifact generated in the working environment.
- [ ] Four explainability visuals and three retention strategies.
- [ ] Batch scoring artifact and tests pass.
- [ ] API health/prediction endpoints and Streamlit UI work.
- [ ] HTML/CSV drift report generated and interpreted.
- [ ] Weekly monitoring schedule documented/configured.
- [ ] Executive summary, presentation, demo script and README included.

8. Known implementation boundary

The supplied handover ZIP includes the code and documentation deliverables. Local raw data, generated model binary, local processed splits, Docker images, and runtime-specific output files may not be bundled. Generate them locally using the README commands. The last user-

run metrics recorded for this project are summarized in `reports/EXECUTIVE_SUMMARY.md`; rerun training to refresh them after any code/data changes.